



AGBE Family

Plateforme de Gestion Familiale (PGF)

UNITÉ · AMOUR · RESPECT · HÉRITAGE

INFRASTRUCTURE

Plan de *déploiement*

Propositions d'hébergement pour une plateforme fluide et scalable : architecture cible, comparatif de trois niveaux de serveurs, dimensionnement, sécurité opérationnelle et feuille de route.

VERSION

1.0 — Sept. 2026

PUBLIC

Décideurs & technique

HORIZON

12–24 mois

Contexte et objectifs

AGBE Family est aujourd'hui rodée en environnement de démonstration : une application Next.js 16 (rendu serveur + API REST), une base SQLite, des fichiers de preuve, et un accès par téléphone mobile pour l'essentiel des membres. La mise en production vise trois objectifs simples à énoncer mais exigeants à tenir dans la durée : une plateforme **fluide** depuis le Togo comme depuis la diaspora, une infrastructure **scalable** capable d'absorber la croissance de la famille (branches, comités, nouveaux registres) sans réécriture, et un coût maîtrisé, adapté à une organisation familiale.

1

Profil d'usage mesuré

- **Effectif initial** : 20 à 50 membres actifs, majoritairement sur smartphone, réseau mobile 3G/4G.
- **Pics de charge** : lancements de campagnes, fin de mois (cotisations), annonces d'événements — quelques dizaines de sessions simultanées au maximum.
- **Données** : base SQLite légère (quelques Mo), preuves de paiement (images/PDF, 8 Mo max par fichier).
- **Exigence de confidentialité** : données financières familiales — hébergement privé, HTTPS obligatoire, sauvegardes fiables.

Ce profil oriente directement l'architecture : à cette échelle, la fluidité dépend davantage de la **qualité de service** (temps de réponse serveur, compression, cache, proximité réseau) que de la puissance brute. La scalabilité, elle, se prépare par la containerisation (Docker) et par une voie de migration claire vers PostgreSQL le jour où plusieurs instances seraient nécessaires. Le plan ci-dessous suit cette logique : partir sur un socle économique et solide, avec un chemin de croissance tracé.

Architecture cible

L'architecture retenue est volontairement sobre : un serveur unique (VPS) porte l'application dans un conteneur Docker, derrière un reverse proxy Nginx qui termine le HTTPS, compresse les réponses et met en cache les ressources statiques. Les données vivent dans deux volumes Docker persistants — la base SQLite et les preuves de paiement — sauvegardés quotidiennement vers un second conteneur dédié¹. Chaque brique est

remplaçable indépendamment : c'est ce découplage, plus que la puissance de la machine, qui apportera la scalabilité future.



Choix assumé : SQLite d'abord, PostgreSQL plus tard. Pour 20–100 membres, SQLite est plus rapide à administrer, sans serveur de base dédié, et sauvegardable par simple copie de fichier. La migration vers PostgreSQL (une source de vérité multi-instances) est documentée au chapitre 06 : elle ne demande pas de réécriture applicative, seulement un changement de fournisseur Prisma et une passe de migration.

CHAPITRE 03

Exigences de fluidité et de sécurité

Les cibles ci-dessous traduisent « fluide » en critères vérifiables. Elles sont volontairement exigeantes pour l'Afrique de l'Ouest, où la latence réseau domine le ressenti : c'est la raison du choix d'un hébergement européen (câbles sous-marins Marseille/Paris — Lomé) associé

à la compression et au cache, plutôt qu'une recherche de proximité géographique impossible à obtenir à coût raisonnable.

Critère	Cible	Moyen mis en œuvre
Latence réseau Lomé ↔ serveur	≤ 150 ms aller-retour	Hébergement Europe de l'Ouest sur backbone câbles
Réponse API (session active)	< 300 ms	Node 22 standalone, SQLite en lecture directe
Premier affichage (pageconnexion)	< 2,5 s en 3G	Code-splitting Next.js, images WebP, gzip
Charge simultanée	50 sessions	2 vCPU / 4 Go — marge ×10 sur le pic attendu
Disponibilité	≥ 99,5 %	Redémarrage Docker automatique, healthcheck, supervision
Confidentialité	HTTPS + privé	Let's Encrypt, rate-limit connexion, données non partagées
Perte de données	≤ 24 h	Sauvegarde quotidienne + copie hors-site hebdomadaire

CHAPITRE 04

Trois niveaux d'hébergement comparés

Trois familles de solutions couvrent le besoin, du plus économique au plus intégré. Les tarifs sont indicatifs (grilles publiques 2026, hors promotions) et se vérifient au moment de la souscription. Pour chaque niveau, nous citons deux fournisseurs de référence afin d'éviter toute dépendance à un opérateur unique.

1

Niveau 1 — VPS économique

Recommandé au départ

Serveur privé virtuel européen, administré par la famille (Docker + Nginx fournis). **Hetzner CX22** (2 vCPU, 4 Go, 40 Go NVMe, 20 To de trafic — Falkenstein/Helsinki) ou **OVHcloud VPS** équivalent (Gravelines/Roubaix). Compte root, image Debian 12.

- **Coût** : $\approx 4\text{--}7$ €/mois — imbattable pour ce niveau de service.
- **Fluidité** : largement suffisante pour 50+ membres simultanés ; latence Lomé $\approx 110\text{--}140$ ms.
- **Scalabilité** : montée verticale immédiate (CX32 : 4 vCPU/8 Go ≈ 9 €) en 10 minutes, sans migration.
- **Contrepartie** : administration à assumer (mises à jour, sauvegardes hors-site, surveillance) — estimée 1–2 h/mois avec les scripts fournis.

2

Niveau 2 — VPS performance

Croissance

Pour absorber une famille élargie (100–300 membres, plusieurs registres actifs, pics de campagnes) : **Hetzner CPX31** (4 vCPU dédiés AMD, 8 Go, 160 Go NVMe) ou **Scaleway GP1-XS** / **DigitalOcean 4 vCPU-8 Go**.

- **Coût** : $\approx 15\text{--}24$ €/mois.
- **Fluidité** : marge $\times 20$ sur les pics ; place pour PostgreSQL local, tableaux de bord lourds et exports simultanés.
- **Scalabilité** : bascule vers PostgreSQL + second conteneur applicatif derrière le load balancer interne de Nginx.
- **Contrepartie** : même administration que le niveau 1, coût $\times 3$.

3

Niveau 3 — Cloud managé / PaaS

Zéro administration

Application et base gérées par le fournisseur : **Railway** ou **Render** (conteneur + PostgreSQL managé), alternative **Fly.io** (déploiement multi-régions). Vercel est écarté : le plan gratuit interdit l'usage commercial et les uploads persistants, et le format standalone actuel est optimisé pour le conteneur.

- **Coût** : $\approx 10\text{--}25$ €/mois selon la consommation (il existe un crédit d'essai à l'inscription).
- **Fluidité** : excellente ; PostgreSQL inclus, sauvegardes automatiques fournies.
- **Scalabilité** : verticale et horizontale en un clic, sans intervention technique.
- **Contrepartie** : coût récurrent croissant avec l'usage, dépendance au fournisseur, exports de preuves à reconfigurer (stockage objet).

	Niveau 1 · VPS éco	Niveau 2 · VPS perf.	Niveau 3 · PaaS
Coût mensuel	4 – 7 €	15 – 24 €	10 – 25 €

Admin. requise	1–2 h/mois	1–2 h/mois	≈ 0
Membres simultanés	50–100	200–300	100+
Base de données	SQLite (volumes)	SQLite → PostgreSQL	PostgreSQL managé
Sauvegardes	Conteneur dédié + hors-site	Idem + snapshots	Fournies (vérifier rétention)
Chemin de croissance	Changement de formule en 10 min	Multi-instances	Intégré
Dépendance fournisseur	Faible (standard)	Faible	Forte (propriétaire)

CHAPITRE 05

Recommandation et dimensionnement

Recommandation : démarrer au Niveau 1 sur Hetzner CX22 (≈ 4,5 €/mois), domaine familial en .com ou .family (≈ 10–15 €/an), Cloudflare gratuit devant Nginx pour le cache statique mondial. Ce socle couvre l'intégralité du besoin actuel avec une marge confortable, pour un budget annuel inférieur à 70 €. La montée au Niveau 2 se décide sur des critères objectifs — pas sur l'intuition — énumérés dans la feuille de route (chapitre 07). Le Niveau 3 reste l'option de repli si aucune ressource technique n'est disponible pour administrer le VPS.



Pourquoi Hetzner en premier choix

- **Rapport puissance/prix** leader du marché européen, trafic 20 To inclus (les preuves de paiement ne seront jamais facturées).
- **Fiabilité** : datacenters certifiés en Allemagne/Finlande, SLA 99,9 %, tableaux de bord de disponibilité publics.
- **Latence Lomé ≈ 110–140 ms** : au niveau des autres hébergeurs européens, la route réseau passant par les câbles sous-marins vers Marseille/Paris.
- **Évolutivité** : passer de CX22 à CX32 se fait par changement de formule et redémarrage — aucune migration de données.
- **Repli simple** : si la famille préfère un interlocuteur francophone, OVHcloud offre une grille équivalente (VPS 2 vCPU/4 Go, datacenters français).

Dimensionnement mémoire vérifié : le serveur Next.js standalone consomme $\approx 150\text{--}250$ Mo en production (mesuré en conditions réelles). Avec 4 Go, la machine porte sans difficulté l'application, Nginx, les sauvegardes et une marge $\times 10$ sur les pics. Le premier facteur limitant à surveiller est le **disque** (preuves de paiement cumulées), d'où le monitoring du volume prévu au chapitre 07.

CHAPITRE 06

Scalabilité : la voie PostgreSQL

SQLite impose une écriture unique — parfait jusqu'à quelques centaines de membres, mais incompatible avec plusieurs conteneurs applicatifs partageant la même base. Le passage à PostgreSQL, prévu dans le schéma Prisma, est l'étape clé du passage à l'échelle. Il ne modifie ni l'API ni l'interface : seul le fournisseur change. La procédure tient en une heure, fenêtre de maintenance comprise.



Procédure de migration (résumé)

- 1. Ajouter un conteneur PostgreSQL 16 au **docker-compose.yml** (volume dédié, réseau interne).
- 2. Dans **prisma/schema.prisma** : **provider = "postgresql"** (l'URL passe par DATABASE_URL, déjà externalisée).
- 3. Exporter les données : script de lecture SQLite → écriture PostgreSQL (les modèles Prisma assurent la compatibilité des types).
- 4. Reconstruire l'image, relancer : **docker compose up -d --build** — l'entrypoint détecte la base existante.
- 5. Sauvegarder l'ancien fichier SQLite (repli possible pendant 30 jours).

Une fois PostgreSQL en place, la scalabilité horizontale devient directe : duplication du service applicatif (Nginx répartit la charge en amont), séparation lecture/écriture, et ajout d'un cache Redis si les tableaux de bord s'alourdissent. Ces étapes ne sont utiles qu'au-delà de 300 membres actifs simultanés — elles sont documentées ici pour montrer que la trajectoire est tracée, pas pour le présent.

Sécurité opérationnelle

La plateforme manipule l'argent et l'intimité de la famille : la sécurité de l'infrastructure est un investissement, pas une option. Les mesures ci-dessous complètent celles déjà intégrées à l'application (mots de passe script, sessions expirantes, journal d'audit, changement de mot de passe forcé) et aux fichiers de déploiement fournis (HTTPS, limite de débit sur la connexion, conteneur sans privilège d'écriture hors volumes).



Mesures de base (à mettre en place dès l'installation)

- **Pare-feu** : seuls les ports 22 (SSH par clé), 80 et 443 sont ouverts (ufw fourni).
- **SSH par clé**, authentification par mot de passe désactivée, éventuellement restriction par adresse IP.
- **Mises à jour automatiques** de sécurité Debian (unattended-upgrades) et veille mensuelle sur les images Docker.
- **Sauvegardes hors-site** : copie hebdomadaire du dernier export SQLite vers un stockage externe (cloud personnel) — le serveur ne doit jamais être la seule copie.
- **Supervision** : vérification de disponibilité externe (service gratuit type UptimeRobot) alertant l'admin en cas d'indisponibilité.



Mesures de renforcement (recommandées à 3-6 mois)

- **Double authentification** pour le compte Admin Général (à intégrer à l'application — évolution logicielle).
- **Scan de vulnérabilités** trimestriel des dépendances (npm audit / advisories GitHub).
- **Journal centralisé** : répliquation des journaux Nginx et applicatifs vers un emplacement en lecture seule.
- **Test de restauration** semestriel : restaurer une sauvegarde sur un environnement de test pour vérifier la procédure.

Feuille de route 12 mois

Le déploiement se joue en trois phases, chacune déclenchée par des critères mesurables plutôt que par le calendrier. Le guide **docs/DEPLOIEMENT.md** du dépôt détaille les commandes de la phase 1, prêtes à copier-coller.

Phase	Contenu	Durée	Déclencheur de sortie
1 • Mise en service	VPS Niveau 1 + domaine + HTTPS + sauvegardes + supervision externe ; création des comptes réels ; retrait des comptes de démonstration.	1 jour	Plateforme accessible, première semaine sans incident.
2 • Rodage	Formation des responsables (guide d'utilisateur), réglages de cache, premier test de restauration, copie hors-site hebdomadaire.	1 mois	Latence et disponibilité dans les cibles du chapitre 03.
3 • Croissance	Montée Niveau 2, migration PostgreSQL, second conteneur applicatif, CDN si trafic international.	au besoin	> 150 membres actifs, OU latence API > 500 ms aux heures de pointe, OU disque > 60 %.

Prochaine action concrète : réserver le VPS Hetzner CX22, pointer le domaine familial sur son adresse IP, puis suivre **docs/DEPLOIEMENT.md** — l'installation complète est une après-midi de travail.



Une fondation solide, prête à grandir.

UNITÉ · AMOUR · RESPECT · HÉRITAGE

Ce plan définit l'infrastructure des douze prochains mois : un socle économique et vérifiable, des critères objectifs de croissance, et une trajectoire complète vers PostgreSQL et la scalabilité horizontale. Les fichiers de déploiement sont déjà dans le dépôt — il ne reste qu'à choisir le serveur.